

レジスタ割り当て — 変数をメモリから追い出す

今日のゴール

局所変数をメモリではなく **callee-saved** レジスタに置く。変数への代入が 5 命令から 1 命令になる。そして「割に合わない昇格」があることを、測って発見する。

1 この回の位置づけ

- 最適化発展シリーズの第 4 回 (2 コマ)。このシリーズでいちばん効果が大きい
- 必須の前提: コマ 16(完成した `mycc.py`) と O1(測定基盤)
- 推奨の前提: O3(命令選択) — 効果を `isel` をかけた状態と比べる
- 編集するのは `regalloc.py` だけ。`mycc.py` は書き換えない

コマ	やること	到達点
1	Step 1~4	<code>golden.py</code> が通り、 <code>fib_rec</code> だけ遅くなることが測れる
2	Step 5~6	<code>fib_rec</code> の悪化が消える

2 ここまで分かっていること

O1 で測ったとおり、いまの出力の 6 割近くが **push/pop** の往復である。なぜそんなに多いのか。変数を 1 つ読むだけで、こういうコードが出ているからである。

```

addi a0, s0, -24      # s のアドレスを計算して
addi sp, sp, -8       # スタックに積んで
sd    a0, 0(sp)
...
ld    a1, 0(sp)       # 右辺を計算して
addi sp, sp, 8         # アドレスを取り出して

```

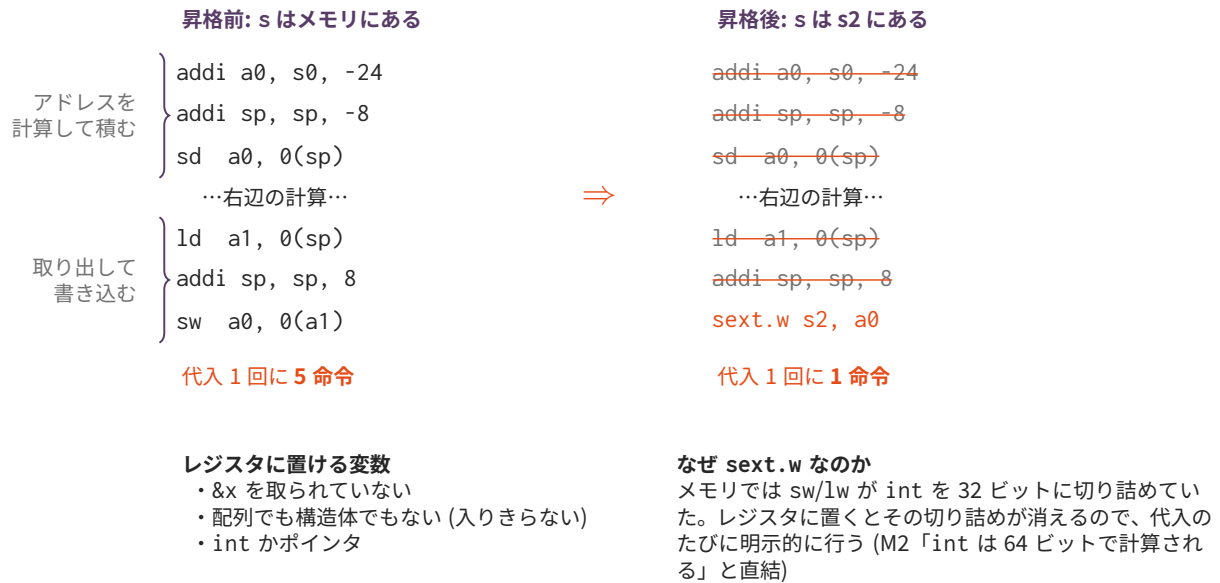


図 1 変数をレジスタに置くと代入が 5 命令から 1 命令になる

```
sw a0, 0(a1) # ようやく書き込む
```

s がレジスタに入っていれば、これは **1 命令**で済む。

3 なぜアセンブリではなくコード生成器を触るのか

03・07 は、出来上がったアセンブリを行単位で書き換えるパスだった。今回はそれができない。

```

int x;
int *p;
p = &x;          /* x のアドレスが要る → x はメモリに実体が必要 */

```

&x が使われているかどうかは **AST** を見ないと分からない。アセンブリになった後では、`addi a0, s0, -24` が「アドレスを取ったから」なのか「値を読むついでに出ただけ」なのか区別できない。

そこで今回は、B2(レジスタスタック) と同じくコード生成器にパッチを当てる。`optcc.py --regalloc` が `regalloc.py` の `patch()` を呼び、`codegen` と `gen_func` を差し替える。

重要

`mycc.py` は 1 行も書き換えない。差し替えは `patch()` の中で完結させる。

4 レジスタに置ける変数

条件	理由
<code>&x</code> を取られていない	アドレスを取るなら、メモリ上に実体が必要
配列でも構造体でもない	レジスタに入りきらない
<code>int</code> かポインタ	8 バイト以内のスカラー

`char` を外しているのは、8 ビットへの切り詰めを 1 命令で書けないためである (基本命令セットに `sext.b` が無い)。

5 なぜ callee-saved(s1~s11) なのか

RV64 の呼び出し規約では、`t0~t6` と `a0~a7` は呼ばれた側が自由に壊してよい (caller-saved)。変数をそこに置くと、関数を 1 回呼ぶだけで値が消える。

`s1~s11` は呼ばれた側が元に戻す義務を負う (callee-saved)。だから呼び出しをまたいでも値が残る。

代わりに、自分がその義務を負う。プロローグで退避し、エピローグで復帰する。

```
main:
    ...
    sd s1, -40(s0)      # 呼び出し元の s1 を預かる
    ...
.L1:
    ld s1, -40(s0)      # 返す前に元へ戻す
    ...
    ret
```

これを忘れると静かに壊れる。自分のテストは通るのに、呼び出し元の変数が書き換わる。原因を追うのが難しい種類のバグである。

6 int の切り詰め — S2 との接続

int の変数は、これまで sw(32 ビット書き込み) と lw(32 ビット読み出し) を通っていた。つまりメモリを通るたびに 32 ビットへ切り詰められていた。

レジスタに置くと、この切り詰めが起きなくなる。S2 で見たとおり、この処理系の int 演算は 64 ビットで行われるので、切り詰めが消えると桁あふれの挙動が変わってしまう。

そこで代入のたびに明示的に切り詰める。

```
sext.w s2, a0      # a0 の下位 32 ビットを符号拡張して s2 へ
```

ポインタは 64 ビットなので mv でよい。

7 実装 (コマ 1)

Step	実装対象	内容
1	escaped_vars	AST を歩いて &x を集める
2	promotable	上の 3 条件で絞る
3	emit_var_read / emit_var_assign	読み書きの生成
4	save_restore	退避と復帰

worth_promoting は、コマ 1 では return list(names)(全部昇格) にしておく。

重要

emit_var_assign では、右辺を **cg.codegen(node.rhs)** で生成する。元の codegen を直接呼ぶと、右辺の部分木にパッチが効かない。

```
found = i;      /* i がレジスタにあるのに、古いメモリから読まれる */
```

fixed15 の f06_break.c がこれを検出する。

8 測ってみると (コマ 1: 全部昇格)

ベンチマーク	静的: 前	後	削減	動的: 前	後	削減	
bubble	281	249	11%	20546	18809	8.5%	
fib_rec	74	76	-3%	69062	73008	-5.7%	← 遅くなった
loop_sum	65	49	24%	7038	5032	28.5%	
matmul	351	299	14%	24546	21544	12.2%	
strops	226	189	16%	69086	59817	13.4%	
合計	997	862	13%	190278	178210	6.3%	

loop_sum は **28.5%** 速くなった。ループの中の代入が 2 つとも 1 命令になったからである。

しかし fib_rec だけ遅くなっている。

9 なぜ fib_rec は遅くなったのか

```
int fib(int n) {
    if (n < 2) { return n; }
    return fib(n - 1) + fib(n - 2);
}
```

n は一度も代入されない。読まれるだけである。

昇格の利得は「代入が安くなること」で稼いでいた。読み出しは、メモリからなら `lw a0, -24(s0)` の 1 命令、レジスタからなら `mv a0, s1` の 1 命令——同じ **1 命令**である。得がない。

一方、費用は確実にかかる。fib は再帰で何万回も呼ばれ、そのたびに `sd s1` と `ld s1` の 2 命令を実行する。

利得：代入 0 回 × 4 命令 = 0
費用：呼び出しのたびに 2 命令

割に合っていない。

10 実装 (コマ 2)

Step	実装対象	内容
5	<code>assign_counts / has_call</code>	代入回数と、呼び出しの有無
6	<code>worth_promoting</code>	費用対効果で選ぶ

呼び出しを含まない関数（葉関数） → 退避が要らない → 全部昇格してよい
呼び出しを含む関数 → 退避の費用がある → 代入される変数だけ昇格する

11 測ってみると (コマ 2)

ベンチマーク	静的：前	後	削減	動的：前	後	削減	
bubble	281	249	11%	20546	18809	8.5%	
fib_rec	74	74	0%	69062	69062	0.0%	← 昇格しない
loop_sum	65	49	24%	7038	5032	28.5%	
matmul	351	299	14%	24546	21544	12.2%	
strops	226	189	16%	69086	59817	13.4%	
合計	997	860	13%	190278	174264	8.4%	

`fib_rec` の悪化が消えた。合計の削減も 6.3% から **8.4%** に増えている。「やらない」という判断も最適化である。

12 まだ残っているもの

割り当て後の `s = s + i` はこうなっている。

```

mv    a0, s2          # s を読む
addi  sp, sp, -8
sd    a0, 0(sp)
mv    a0, s1          # i を読む
ld    a1, 0(sp)
addi  sp, sp, 8
add   a0, a1, a0
sext.w s2, a0         # s へ書く

```

add の入力は結局 s2 と s1 である。add a0, s2, s1 と書ければ、前の mv 2 つと push/pop が丸ごと要らなくなる。

それが O6(コピー伝播と死コード除去)の仕事である。この回が、次の回の材料を作っている。

13 発展課題

1. **char** も昇格する: 8 ビットへの切り詰めをどう書くか。slli + srar の 2 命令で書けるが、それでも得か
2. **caller-saved** を使い分ける: 呼び出しをまたがない変数は t0 ~ t6 に置けば退避が要らない。「またぐかどうか」はどう判定するか (**O5** の生存解析)
3. **12 個以上の変数**: 局所変数が 11 個を超える関数を書くとき、レジスタが足りなくなる。いまは宣言順に先着順で割り当てている。どれを優先すべきか
4. **worth_promoting** の閾値: いまは「代入が 1 回でもあれば」という単純な判定である。ループの中の代入を重く見る (深さ × 10 など) と結果は変わるか
5. **B2 と組み合わせる**: B2(レジスタスタック) は一時値を、O4 は変数をレジスタに置く。両方かけるとどうなるか

補足

コラム: 本物のレジスタ割り当ては「塗り分け」問題である。教科書 (タイガーブック 11 章) のレジスタ割り当ては、**干渉グラフ** (同時に生きている変数どうしを辺で結んだグラフ) を作り、隣どうしが同じ色にならないように **k 色で塗り分ける** (グラフ彩色)。色がレジスタに対応し、塗れない変数はメモリへ追い出す (スピル)。

今回は「変数 1 つにレジスタ 1 つ」を先着順で配ったので、彩色は要らなかった。干渉グラフを作るには「どの変数がどこで生きているか」が必要で、それを求めるのが **O5** の生存変数解析である。